

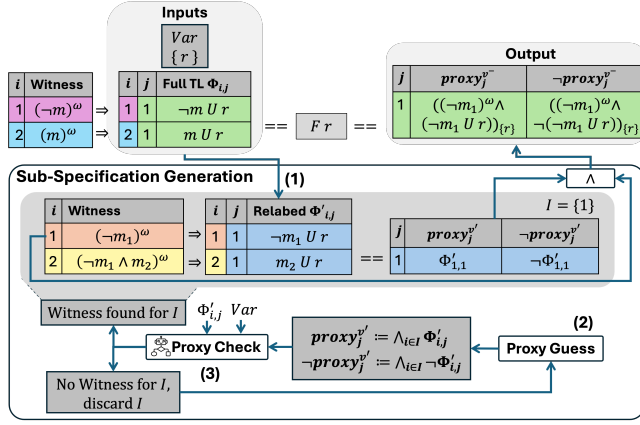


Figure 10: Example of Sub-Specification Generation. Colors denote index i and j correspondence. Sub-Specification Generation uses Proxy Guess to guess a standard proxy and Proxy Check to verify it satisfies Eq. 1. Proxy Guess formulates standard proxies as conjunctions of full specifications to capture their common behavior. SSG outputs the standard proxy $proxy_j^{v-}$ as a conjunction of a subset of full specifications $\Phi_{i,j}$ projected to the variables in Var (i.e., the variables used in the sub-requirement j). $(\phi)_{Var}$ indicates a projection of LTL formula ϕ with respect to the variables in the set Var . Specifically, all satisfying traces of $(\phi)_{Var}$ only use variables in Var , and other all other variables are omitted. Fr is shown for illustrative purposes and does not impact SSG. All traces that satisfy Fr also satisfy $proxy_j^{v-} = ((\neg m_1)^\omega \wedge (\neg m_1 U r))_{\{r\}}$. All traces that satisfy $\neg(Fr)$ also satisfy $\neg proxy_j^{v-} = ((\neg m_1)^\omega \wedge \neg(\neg m_1 U r))_{\{r\}}$. For each i , there exists a witness trace for which the full specification $\Phi_{i,j}$ is equivalent to Fr . Since $proxy_j^{v-}$ is equivalent to Fr , given the same witness trace for each i , the full specification $\Phi_{i,j}$ is also equivalent to $proxy_j^{v-}$. Thus, $proxy_j^{v-}$ as a standard proxy satisfies Eq. 1

A Automatic Proxy Generation

Given a canonicalized node v^- , for each of its structured sub-requirements, ARTEMIS's Sub-Specification Generation (SSG) procedure produces standard proxies, from which abstract proxies are derived.

To generate proxies, one may consider searching all possible TL formulas until finding ones where Proxy Check identifies a witness trace (§5.2). However, this is likely impractical since there are an infinite number of TL formulas. Instead, ARTEMIS's SSG derives proxies in terms of the given full specifications $\Phi_{i,j}$.

Fig. 10 illustrates the steps of SSG on an example input for some node v^- . SSG takes as input the set of full specifications $\Phi_{i,j}$ and the set of variables Var that are used in the structured sub-requirements of v^- (i.e., the set of variables the proxies use). SSG first relates every variable not in Var in each full specification $\Phi_{i,j}$ with an index tag i to produce $\Phi'_{i,j}$ (step 1 in Fig. 10), ensuring that those variables appear only in the i -th specification. Then, Proxy Guess derives the

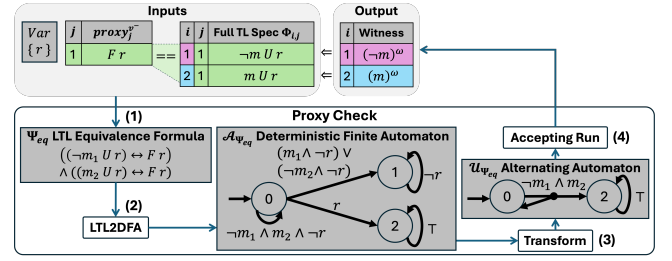


Figure 11: Proxy Check produces a set of witness traces showing that proxies satisfy Eq. 1. It uses a specialized automata transform to obtain a trace agnostic to the variables used by the proxies. Colors denote index i and j correspondence.

proxies in terms the full specifications (as a conjunction) to capture common allowed/disallowed behavior among them (step 2 in Fig. 10): $proxy_j^{v-} := \bigwedge_{i \in I} \Phi'_{i,j}$; $\neg proxy_j^{v-} := \bigwedge_{i \in I} \neg \Phi'_{i,j}$ such that I is a minimal set of indices of the parent node v^+ sub-requirements that enables the final output proxies to satisfy Eq. 1. Observe that when I contains all indices, this derivation of proxies always satisfies Eq. 1. Thus, SSG always produces proxies that satisfy Eq. 1 if they exist. Proxy Guess proposes a possible I and then Proxy Check produces a witness trace (step 3 in Fig. 10). Proxy Guess minimizes $|I|$ because doing so minimizes formula size of the proxies and in turn minimizes trace generation runtime.

If Proxy Check does not identify a witness trace for an I , then I is discarded from the search and another I is proposed until producing a witness trace. Identifying a witness trace indicates a valid proxy is found (w.r.t Eq. 1). Since Proxy Guess formulates proxies as conjunctions of full specifications, SSG outputs those proxies constrained to the witness trace (see output of Fig. 10), which only constrains the variables not in Var (i.e., the variables not used by the proxies). When generating traces using SSG proxies, ARTEMIS post-processes them by omitting all variables not in Var (denoted by $(\phi)_{Var}$ in Fig. 10). In Fig. 10, observe that the output proxy is equivalent to Fr with respect to variable r , and that Fr is shown to be a valid proxy (satisfies Eq. 1) in Fig. 11.

SSG can be extended support other structured decomposition approaches (e.g., SYNTHL §2.3), but doing so requires re-generating proxies for each set of candidates per requirement.

Proxy Check. For a canonicalized node v^- , ARTEMIS's Proxy Check procedure verifies that a set of standard proxies for v^- satisfies Eq. 1. Proxy Check takes as input a set of LTL formulas encoding one proxy $proxy_j^{v-}$ for each j candidate of v^- , a set of variables Var that are used in v^- 's candidates, and a set of LTL formulas encoding all full specifications $\Phi_{i,j}$ produced from combining the i th sub-requirement of parent node v^+ with the j th sub-requirement of child node v^- . For each i , Proxy Check outputs a witness trace $t_i^{witness}$ over the variables not in Var , for which all proxies $proxy_j^{v-}$ and the full TL specification $\Phi_{i,j}$ are equivalent (as defined in Eq. 1). If no witness traces exists for some proxies, they cannot satisfy the definition of a proxy (Eq. 1), and Proxy Check returns $\emptyset$.

A standard equivalence encoding between $proxy_j^{v-}$ and $\Phi_{i,j}$ in a satisfiability model checker query is insufficient to derive $t_i^{witness}$

of Eq. 1, as it yields traces over all variables of all $\Phi_{i,j}$. To generate $t_i^{witness}$ independent from the valuations of variables in Var , ARTEMIS introduces the Proxy Check procedure. Note that the abstraction constraint for abstract proxies, Eq. 2, can be checked via a standard LTL model checker query.

Fig. 11 gives an example of Proxy Check. Observe that the output witness traces show the proxies satisfy Eq. 1: $(\neg m)^\omega \Rightarrow (Fr \leftrightarrow \neg mUr)$ holds and $(m)^\omega \Rightarrow (Fr \leftrightarrow mUr)$ holds.

Proxy Check first relabels every variable not in Var in each $\Phi_{i,j}$ with an index tag i to produce $\Phi'_{i,j}$, ensuring that those variables appear only in the i -th specification. Then Proxy Check constructs an LTL equivalence formula Ψ_{eq} that encodes equivalence between $proxy_j^{v^-}$ and $\Phi'_{i,j}$: $\Psi_{eq} = \bigwedge_{i,j} (proxy_j^{v^-} \leftrightarrow \Phi'_{i,j})$ (step 1, Fig. 11). Then, Proxy Check transforms Ψ_{eq} into a deterministic automaton [8, 30] $\mathcal{A}_{\Psi_{eq}}$ (step 2, Fig. 11), which represents all satisfying traces to Ψ_{eq} . A satisfying trace of Ψ_{eq} is an accepting run of $\mathcal{A}_{\Psi_{eq}}$.

Then, to produce the output witness trace $t_i^{witness}$ for each i , Proxy Check deploys a specialized transformation of $\mathcal{A}_{\Psi_{eq}}$ into an alternating automaton [30] $\mathcal{U}_{\Psi_{eq}}$ (step 3, Fig. 11), whose accepting runs are $t_i^{witness}$. Alternating automata are well-suited for this task because their parallel runs can be used to encode every valuation of variables in Var satisfying Ψ_{eq} , yielding witness traces that satisfy Ψ_{eq} regardless of the valuations of variables in Var . In an alternating automaton, a universal edge (i.e., one source leading to multiple destinations) spawns parallel runs, and a run is accepting only if every spawned run satisfies the automaton's acceptance condition.

To produce $\mathcal{U}_{\Psi_{eq}}$, Proxy Check first discards all states in $\mathcal{A}_{\Psi_{eq}}$ that do not cover every valuation of variables in Var . Then, for each

remaining state, Proxy Check merges outgoing edges that exhibit differing valuations of variables in Var and matching valuations of variables not in Var into a single universal edge.

For each i , Proxy Check outputs witness trace $t_i^{witness}$ by extracting the relevant variables in an accepting run (step 4, Fig. 11). For example, in Fig. 11, $\mathcal{U}_{\Psi_{eq}}$ yields trace $(\neg m_1 \wedge m_2)^\omega$, where m_1 corresponds to $i = 1$ and m_2 corresponds to $i = 2$.

Offline Proxy Generation. Instead of regenerating proxies for every decomposition tree, SSG generates the standard proxy for each sub-requirement once *offline*. Because Structured NLs express a finite set of TL specifications, for a structured NL, we can derive a standard proxy for a sub-requirement that captures common behavior among all full candidate specifications that could ever contain the sub-requirement. In particular, we enumerate the finitely many possible canonicalized decomposition trees whose candidates consist of all possible instantiations of structured NL fields (§2.2) where each tree expresses all possible full specifications of the structured NL (modulo variables names and boolean expressions). Then, SSG derives a standard proxy for every sub-requirement in the child node of all canonicalized trees. Since any decomposition tree generated by an LLM would express a subset of all possible full specifications the structured NL expresses, the offline-generated proxies always capture common behavior among full specifications that contain it. After this offline step, to obtain the proxy for some structured sub-requirement in any decomposition tree, ARTEMIS canonicalizes the tree with respect to its node, and instantiates the boolean expressions (along with variable names) of the sub-requirement into an offline-generated proxy.